WOXSEN UNIVERSITY

SCHOOL OF TECHNOLOGY

DATABASE MANAGEMENT SYSTEMS

PROJECT-BASED LEARNING (PBL) REPORT

PROJECT 39

DESIGN AND IMPLEMENTATION OF A DATABASE MANAGEMENT SYSTEM FOR IT HELPDESK AND ASSET SUPPORT MANAGEMENT SYSTEM

Submitted by

Student Name	Kothuru Shiva Ganesh
Roll Number	25WU0105009
Program / Section	B.Tech CSE AIML Rhinos

ACKNOWLEDGEMENT

I express my sincere gratitude to Woxsen University, School of Technology, for providing the opportunity to undertake this Project-Based Learning activity as part of the Database Management Systems course. The project provided a practical setting for applying relational modelling, key and integrity-constraint design, SQL querying, normalization and application-to-database connectivity.

I thank the course faculty for the guidance and academic framework provided through the project brief and report template. The prescribed milestones helped structure the work from problem identification and conceptual modelling to logical design, implementation, testing and documentation.

I also acknowledge the official documentation for SQLite, Python and Flask, and the database systems reference material consulted while preparing the implementation and report. AI-assisted tools were used for selected development and documentation tasks; the resulting project artifacts and statements must be reviewed by the submitting student and disclosed in accordance with university policy.

Finally, I acknowledge the value of hands-on testing of the local prototype, including ticket creation, assignment, status changes, resolution, asset search, maintenance records and CSV reporting. I remain responsible for checking the final document, completing the student-specific fields and ensuring that the submitted work follows the university's academic requirements.

UNDERTAKING

I hereby submit the project entitled “Design and Implementation of a Database Management System for IT Helpdesk and Asset Support Management System” as part of the Project-Based Learning activity for the Database Management Systems course.

I confirm that I have reviewed the database design, SQL scripts, application behavior, screenshots, test evidence and references included in this report. I will verify that all technical descriptions match the submitted repository, complete the missing student and faculty information, and acknowledge external resources or AI-assisted tools as required by university policy.

I understand that the sample users, assets, tickets, maintenance records and reports shown in the prototype are synthetic demonstration data. The application is an academic prototype and is not represented as a production helpdesk system.

Student signature

Name: Kothuru Shiva Ganesh

Roll Number: 25WU0105009

Program: B.Tech CSE (AI & ML)

Date : 10/10/2026

TABLE OF CONTENTS

Acknowledgement	2
Undertaking	3
List of Tables	5
List of Figures	6
List of Acronyms	7
Abstract and Keywords	8
CHAPTER 1 PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM	9
1.1 Problem Statement	9
1.2 Objectives	9
1.3 Scope and Assumptions	9
1.4 Proposed Solution Strategy	10
1.5 Functional Requirements	10
1.6 Entity-Relationship Diagram	11
1.7 Entity and Relationship Summary	12
CHAPTER 2 LOGICAL DATABASE DESIGN	13
2.1 Relational Schema	13
2.2 Keys and Integrity Constraints	13
2.3 Functional Dependencies	14
2.4 Normalization up to 3NF	15
2.5 Data Dictionary	16
2.6 DDL and Sample Data	20
CHAPTER 3 APPLICATION DEVELOPMENT	22
3.1 Application Overview	22
3.2 Development Environment and Database Connectivity	23
3.3 Application Modules and Database Mapping	24
3.4 CRUD Operations	24
3.5 Business Rules, Validation and Error Handling	25
3.6 Reports and Database Operations	25
3.7 Application Screens	26
3.8 Testing and Results	32
CHAPTER 4 CONCLUSION	33
4.1 Limitations and Future Enhancements	33
References	34
Appendix A – Complete Database Script	35
Appendix B – Application Source Code	36
Appendix C – Additional Test Data and Outputs	37

LIST OF TABLES

Table 1. Scope and Assumptions 9

Table 2. Proposed Solution Strategy 10

Table 3. Functional Requirements 10

Table 4. Entity and Relationship Summary 12

Table 5. Relational Schema 13

Table 6. Keys and Integrity Constraints 14

Table 7. Normalization Status up to 3NF 15

Table 8. Data Dictionary 16

Table 9. Database Implementation Environment 20

Table 10. Application Development Environment 23

Table 11. Application Module-to-Database Mapping 24

Table 12. CRUD Operations 24

Table 13. Business Rules, Validation and Error Handling 25

Table 14. Reports and Database Operations 25

Table 15. Test Cases 32

LIST OF FIGURES

Figure 1. Entity-Relationship Diagram of the Proposed System 11

Figure 2. Application and Data Architecture 23

Figure 3. Overview Dashboard 26

Figure 4. New Ticket Form with Incident Details 26

Figure 5. Ticket HD-26013 Created Successfully 27

Figure 6. Ticket Status Updated to In Progress 27

Figure 7. Technician Assignment Confirmation 28

Figure 8. Ticket HD-26013 Resolved with Recorded Resolution 28

Figure 9. Asset Inventory 29

Figure 10. Asset Search Result for AST-1001 29

Figure 11. Asset Status Filter: In Repair 30

Figure 12. Maintenance Schedule and Recorded Costs 30

Figure 13. Reports and Analytics Dashboard 31

Figure 14. Exported Ticket CSV Report 31

LIST OF ACRONYMS

Acronym	Expansion
CRUD	Create, Read, Update and Delete
CSV	Comma-Separated Values
DBMS	Database Management System
DDL	Data Definition Language
ERD	Entity-Relationship Diagram
FK	Foreign Key
HTML	HyperText Markup Language
HTTP	Hypertext Transfer Protocol
PK	Primary Key
SLA	Service-Level Agreement / target
SQL	Structured Query Language
3NF	Third Normal Form
UI	User Interface
URL	Uniform Resource Locator

ABSTRACT

IT departments often manage support requests, equipment ownership, warranty dates and maintenance records across disconnected emails or spreadsheets. This fragmentation makes ticket status difficult to track, weakens links between incidents and affected assets, and limits visibility into service deadlines, technician workload and recurring faults. This project designs and implements a relational database-backed IT Helpdesk and Asset Support Management System to centralize ticket and asset information and support traceable service workflows. The conceptual ER model is mapped to a 15-table SQLite schema using primary and foreign keys, unique constraints, CHECK and DEFAULT constraints, and indexes. Separate relations represent users, departments, assets, ticket reference values, incident and service-request details, assignments, status histories, resolutions, warranties and maintenance. A Python Flask web application provides ticket creation, technician assignment, status updates, resolution recording, asset search and filtering, maintenance tracking, operational reports, and ticket CSV export. Manual browser demonstrations verified the creation, assignment, status transition and resolution of ticket HD-26013, as well as asset filtering, report rendering and CSV download. The prototype uses synthetic data and is intended for academic evaluation; automated test-suite execution remains to be verified. The project illustrates how normalized relational design and application validation can improve consistency and traceability in helpdesk records.

KEYWORDS

IT service management, relational database, entity-relationship modelling, Third Normal Form, SQLite, ticket lifecycle

CHAPTER 1
PROBLEM STATEMENT, SOLUTION STRATEGY AND ER DIAGRAM

1.1 Problem Statement

An IT helpdesk receives incidents such as network failures, hardware faults and application errors, as well as service requests such as software installation or account setup. If requests are tracked through email threads and separate spreadsheets, the current status, priority, responsible technician, related asset, warranty coverage and resolution history may be distributed across multiple locations. Staff may need to ask for repeated updates, duplicate records may be created, and it can be difficult to find older incidents associated with a particular device or to identify common issue categories.

The project addresses this information-management problem through a normalized relational database and a connected web application. The system records shared ticket attributes once and stores type-specific incident or service-request fields in related subtype tables. Department, user, asset, priority and category reference data are stored independently. Assignment events, status changes, resolutions, warranties and maintenance activities are represented as related records so that the database can support operational views without packing repeating events into a single ticket or asset row. The delivered prototype uses synthetic data and demonstrates local academic workflows rather than serving as a production helpdesk [1].

1.2 Objectives

- Design a conceptual ER model covering departments, users, assets, warranties, tickets, support staff, assignments, status history, resolutions and maintenance.
- Convert the approved model into a relational SQLite schema with primary keys, foreign keys, uniqueness, nullability, CHECK and DEFAULT constraints, and useful indexes.
- Normalize the core relations up to Third Normal Form by separating reference values, user identities, technician-specific information and repeating history records.
- Develop a Flask application that performs ticket and asset CRUD operations, assignment and status workflows, resolution recording, maintenance updates and report generation against the database.
- Demonstrate representative database queries, search/filter behavior, CSV export and positive/negative validation scenarios, with manual results distinguished from automated test results.

1.3 Scope and Assumptions

Table 1. Scope and Assumptions

Item	Project-specific description
In scope	Create incidents and service requests; generate ticket numbers and SLA due times; link optional assets; assign technicians; record status history and resolutions.
In scope	Register/edit/search assets; track department and user allocation; view warranty dates and related ticket/maintenance history.
In scope	Record preventive/corrective maintenance, schedules, completion dates, status and non-negative cost values; show dashboard and reports; export ticket data to CSV.

Item	Project-specific description
Out of scope	Production login and identity integration, role-based authorization, email notifications, cloud hosting, real-time monitoring and integration with enterprise tools.
Assumptions	The project uses local synthetic data for demonstration; the initial database is created and seeded locally; one administrator identity is used to attribute prototype actions.
Assumptions	SQLite is the implemented runtime DBMS. The supplied MySQL 8 schema is a reference for a possible migration, not the current application connector.

1.4 Proposed Solution Strategy

Table 2. Proposed Solution Strategy

Solution element	Project-specific description
Primary users / roles	Requesters create or track tickets; support technicians receive assignments, update work and record resolutions; the seeded administrator demonstrates global operations. Authentication is not implemented.
Major modules	Overview dashboard; tickets; asset inventory and asset details; technician directory; maintenance; global search; reports and CSV export.
Workflow	A ticket is created with requester, type, category, priority and optional asset. The app calculates an SLA due timestamp, creates an incident/service-request subtype row and a first status-history event. A technician can be assigned, status changed, and resolution documented.
Business rules	Unique ticket numbers and asset tags; controlled status values; transition checks in application logic; resolution text required before resolving; valid date and non-negative-cost checks; foreign-key validation.
Outputs	Open-ticket ageing, overdue work, first-assignment timing, technician load, asset history, recurring categories, warranty-expiry view, maintenance totals and CSV export.
Implementation pattern	Flask route handlers use request-scoped SQLite connections, `sqlite3.Row`, SQL parameter placeholders and transaction contexts for multi-row changes.

1.5 Functional Requirements

Table 3. Functional Requirements

Req. ID	Functional requirement	Priority
FR-01	Create incident/service-request tickets with title, description, requester, category and priority.	High
FR-02	Generate a unique human-readable ticket number and derive a due timestamp from configured priority SLA hours.	High
FR-03	Optionally link a ticket to an asset and show requester, department, category and linked-device information.	High
FR-04	Assign or reassign available support staff and preserve assignment history.	High
FR-05	Validate allowed ticket status transitions and preserve status-history events with timestamp and note.	High
FR-06	Record resolution details and technician attribution before marking a ticket Resolved.	High
FR-07	Register, edit, search and filter assets using asset tag, model, serial number, department or status.	High
FR-08	Store warranty and maintenance history for each asset and validate dates/costs.	Medium
FR-09	Show technician load, ticket ageing, SLA due information, recurring categories and warranty-expiry views.	Medium
FR-10	Export ticket report data as a CSV file with key ticket fields.	Medium

1.6 Entity-Relationship Diagram (ERD)

Figure 1 shows the conceptual relationship structure used by the implemented schema. Reference data is separated from operational records; tickets share common attributes and have optional one-to-one subtype records; assignments and status history are one-to-many event relations; assets can have multiple warranties, linked tickets and maintenance records. The diagram source is stored in `docs/ER_DIAGRAM.mmd`; the repository's SQLite DDL remains the final authority for exact column and constraint details [2].

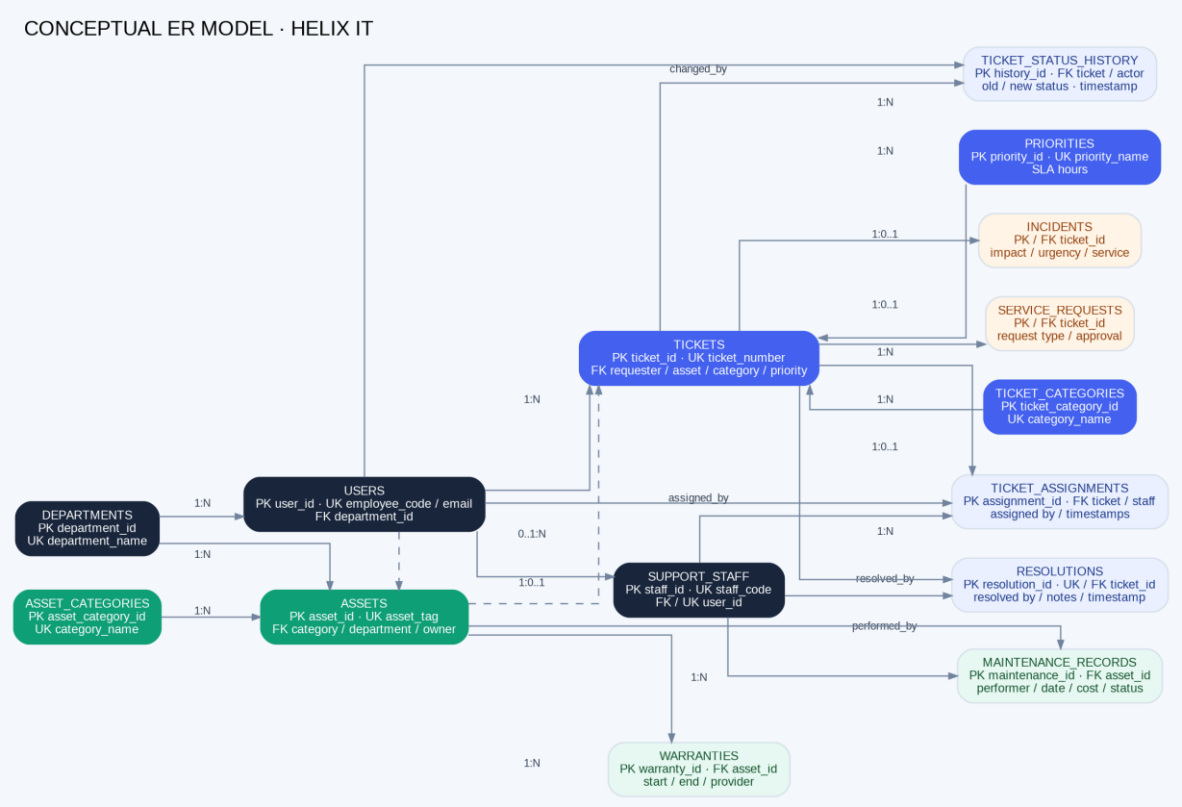


Figure 1. Entity-Relationship Diagram of the Proposed System

1.7 Entity and Relationship Summary

Table 4. Entity and Relationship Summary

Entity	Primary key	Purpose / relationship
departments	department_id	One department has many users and assets.
users	user_id	User identity belongs to one department; users can request tickets and record audit events.
asset_categories	asset_category_id	A category classifies many assets.
assets	asset_id	Asset belongs to one category and department; may be assigned to a user and linked to tickets, warranties and maintenance.
warranties	warranty_id	Each warranty belongs to one asset; an asset may have multiple records over time.
ticket_categories	ticket_category_id	A category classifies many tickets.
priorities	priority_id	A priority provides SLA hours and display order for many tickets.
tickets	ticket_id	Each ticket has one requester, priority and category and may reference an asset.
incidents	ticket_id	Optional one-to-one subtype for incident impact, urgency and affected service.
service_requests	ticket_id	Optional one-to-one subtype for request type and approval metadata.
support_staff	staff_id	One user can have one support-staff profile; technicians can receive many assignments.
ticket_assignments	assignment_id	Timestamped assignment history; multiple events may relate to one ticket and staff member.
ticket_status_history	history_id	Stores multiple state-change events per ticket and the user attributed to each event.
resolutions	resolution_id	At most one current resolution row per ticket; references the resolving technician.
maintenance_records	maintenance_id	Each maintenance record belongs to one asset and may reference a performing technician.

CHAPTER 2
LOGICAL DATABASE DESIGN

2.1 Relational Schema

Table 5. Relational Schema

Relation	Primary key	Foreign-key relationships	Purpose
departments	department_id	—	Department reference data
users	user_id	department_id → departments	User identities and roles
asset_categories	asset_category_id	—	Controlled asset type labels
ticket_categories	ticket_category_id	—	Controlled support area labels
priorities	priority_id	—	Priority label, ordering and SLA hours
assets	asset_id	assigned_user_id → users; department_id → departments; asset_category_id → asset_categories	Asset master and allocation
warranties	warranty_id	asset_id → assets	Warranty coverage and expiry
tickets	ticket_id	priority_id → priorities; ticket_category_id → ticket_categories; asset_id → assets; requester_id → users	Shared incident/request fields
incidents	ticket_id	ticket_id → tickets	Incident-only impact and urgency
service_requests	ticket_id	ticket_id → tickets	Service-request subtype fields
support_staff	staff_id	user_id → users	Technician profile and specialisation
ticket_assignments	assignment_id	assigned_by → users; staff_id → support_staff; ticket_id → tickets	Assignment/reassignment events
ticket_status_history	history_id	changed_by → users; ticket_id → tickets	Ticket status audit events
resolutions	resolution_id	resolved_by → support_staff; ticket_id → tickets	Resolution and preventive-action details
maintenance_records	maintenance_id	performed_by → support_staff; asset_id → assets	Scheduled/completed asset service

2.2 Keys and Integrity Constraints

The primary key of each table is a surrogate integer identifier except the one-to-one subtype relations `incidents` and `service_requests`, which use `ticket_id` as both primary key and foreign key. Natural identifiers are protected by UNIQUE constraints where required. Foreign keys prevent orphan records when a referenced requester, category, asset, staff member or ticket is missing. The application enables SQLite foreign-key enforcement on each request connection. Referential actions are selected according to the intended lifecycle: for example, deleting a ticket cascades its subtype/history records, while referenced requester/category/department rows are restricted; a removed assigned user or ticket-linked asset is set to NULL where the schema permits [3], [4].

Table 6. Keys and Integrity Constraints

Constraint type	Tables / attributes	Implemented rule or example	Purpose
Primary key	All 15 tables	Surrogate integer key; `ticket_id` is the key in both subtype tables.	Identifies one relation row.
Unique business identifiers	users, assets, tickets, support_staff, reference tables	Employee code, email, asset tag, optional serial, ticket number, staff code and lookup names/orders.	Prevents duplicate identity/tag/reference values.
Foreign keys	users, assets, tickets, warranties, assignment/history, resolutions, maintenance	Reference parent PK values with ON UPDATE / ON DELETE rules.	Maintains relationship integrity.
NOT NULL	Required identity, classification and workflow fields	Names, required references, ticket text, maintenance date/type and resolution details.	Prevents missing mandatory values.
CHECK: enumerations	users, assets, priorities, tickets, incidents, requests, maintenance, history	Role, asset status, priority, ticket type/status, impact/urgency, approval/status labels.	Constrains values to supported states.
CHECK: date/number rules	assets, tickets, warranties, assignments, maintenance, priorities	Valid ISO date values; warranty end >= start; assignment end >= start; SLA > 0; cost >= 0.	Rejects invalid date/cost/range values.
CHECK: text lengths	tickets, resolutions	Trimmed ticket title > 2 chars; description > 4 chars; resolution >= 8 chars.	Prevents empty or too-short core descriptions.
DEFAULT	users, assets, departments, tickets, priorities-related workflow and history tables	Examples include role=Requester, asset status=In Use, location=Main Campus, status=Open and timestamps.	Provides stable default values when omitted.
Application-level constraints	`app.py` ticket routes	Allowed status transitions, resolution-before-close checks and form validation.	Enforces workflow rules spanning multiple tables.

2.3 Functional Dependencies

Functional dependencies explain why the selected keys determine the non-key attributes. In the following representative dependencies, "→" means that a value on the left uniquely determines the corresponding value(s) on the right. These dependencies are derived from the actual primary and unique constraints in the SQLite schema.

- `department_id → department_name, location`; because `department_name` is UNIQUE, `department_name → department_id, location` as well.
- `user_id → employee_code, full_name, email, phone, department_id, role, is_active, created_at`; employee_code and email are alternate unique determinants.
- `asset_id → asset_tag, asset_name, asset_category_id, department_id, assigned_user_id, manufacturer, model_number, serial_number, purchase_date, asset_status, location, notes, created_at`; asset_tag is a candidate key, and serial_number is a candidate key when supplied.
- `ticket_id → ticket_number, ticket_type, requester_id, asset_id, ticket_category_id, priority_id, title, description, status, created_at, updated_at, due_at, resolved_at`; `ticket_number` is a unique alternate identifier.
- `priority_id → priority_name, sla_hours, sort_order`; priority_name and sort_order are also unique.
- `ticket_id → impact, urgency, occurred_at, affected_service` in `incidents`; `ticket_id → request_type, approval_status, requested_for` in `service_requests`.
- `staff_id → user_id, staff_code, specialization, shift_name, is_available`; both user_id and staff_code are unique in `support_staff`.
- `assignment_id → ticket_id, staff_id, assigned_by, assigned_at, unassigned_at, assignment_note`; separate assignment rows preserve repeated assignments rather than storing a repeating list on `tickets`.

- `history_id → ticket_id, old_status, new_status, changed_by, changed_at, note`; history records are event facts and are not collapsed into the current ticket status.
- `resolution_id → ticket_id, resolved_by, resolution_text, root_cause, resolved_at, preventive_action`; `ticket_id` is UNIQUE in `resolutions`, so it is an alternate determinant.
- `maintenance_id → asset_id, performed_by, maintenance_type, scheduled_date, completed_date, cost, status, notes, created_at`; each row represents one lifecycle event.

2.4 Normalization up to Third Normal Form (3NF)

The schema is designed to satisfy the requirements of 1NF, 2NF and 3NF for the listed relations. The justification below is based on atomic attributes, the keys declared in the DDL and the separation of descriptive reference data from transactional facts. This is a design analysis; it does not claim a formal BCNF proof [1].

- First Normal Form (1NF): each column stores a single value from its domain. Assignment events, status changes, warranties and maintenance history are represented as separate rows, not as delimited lists or repeating groups.
- Second Normal Form (2NF): each implemented relation has a single-column primary key. Consequently, there is no partial dependency on only part of a composite primary key. The ticket subtype relations use ticket_id as their sole key.
- Third Normal Form (3NF): non-key descriptive facts are stored with their own determinants. Department names are stored in departments, user attributes in users, SLA values in priorities, classification labels in lookup tables, and technician-specific attributes in support_staff. Ticket assignment/status events and resolution/maintenance information are separated from the parent ticket/asset master.
- BCNF: not separately assessed in this project. The submission target is normalization through 3NF, consistent with the stated project deliverables.

Table 7. Normalization Status up to 3NF

Relation	1NF	2NF	3NF	BCNF	Rationale / decomposition
departments	Yes	Yes	Yes	Not assessed	Unique department name determines location; descriptive department data kept once.
users	Yes	Yes	Yes	Not assessed	Department is referenced by ID; identity attributes are not repeated in tickets.
asset_categories	Yes	Yes	Yes	Not assessed	Category label and description stored in lookup relation.
ticket_categories	Yes	Yes	Yes	Not assessed	Support category values stored once.
priorities	Yes	Yes	Yes	Not assessed	SLA hours depend on priority reference, not repeated per ticket.
assets	Yes	Yes	Yes	Not assessed	User, department and category represented by keys; tag identifies asset.
warranties	Yes	Yes	Yes	Not assessed	Warranty facts are separate per asset and period/provider.
tickets	Yes	Yes	Yes	Not assessed	Common ticket facts use IDs for requester, category, priority and optional asset.
incidents	Yes	Yes	Yes	Not assessed	Ticket subtype fields depend on ticket_id.
service_requests	Yes	Yes	Yes	Not assessed	Request subtype fields depend on ticket_id.
support_staff	Yes	Yes	Yes	Not assessed	Technician specialisation/shift separated from general user identity.
ticket_assignments	Yes	Yes	Yes	Not assessed	Repeated assignment events use one row per event.
ticket_status_history	Yes	Yes	Yes	Not assessed	Status events use one row per change.
resolutions	Yes	Yes	Yes	Not assessed	Resolution facts stored separately; ticket_id is unique in this table.
maintenance_records	Yes	Yes	Yes	Not assessed	Each maintenance action is a separate event for an asset.

2.5 Data Dictionary

Table 8 summarises every column defined in the SQLite DDL, including the declared storage type, principal constraints and functional purpose. SQLite uses dynamic typing with declared type affinity; the types listed here are those declared in the schema file. Foreign-key references and CHECK constraints are derived from `schema_sqlite.sql`.

Table 8. Data Dictionary

Table.Attribute	SQLite type	Key / constraint	Description
departments.department_id	INTEGER	PK	Surrogate identifier for the department.
departments.department_name	TEXT	NOT NULL; UNIQUE	Unique department name.
departments.location	TEXT	NOT NULL; DEFAULT 'Main Campus'	Department location; defaults to Main Campus.
users.user_id	INTEGER	PK	Surrogate identifier for a person or system user.
users.employee_code	TEXT	NOT NULL; UNIQUE	Unique employee or user code.
users.full_name	TEXT	NOT NULL	Display name used in requester, assignment and audit views.
users.email	TEXT	NOT NULL; UNIQUE	Unique email address for the synthetic demo identity.
users.phone	TEXT	Nullable	Optional contact phone number.
users.department_id	INTEGER	NOT NULL; FK -> departments.department_id	Required department membership.
users.role	TEXT	NOT NULL; CHECK: Admin/Requester/Technician; DEFAULT 'Requester'	Admin, Requester or Technician; defaults to Requester.
users.is_active	INTEGER	NOT NULL; CHECK: 0 or 1; DEFAULT 1	Boolean-like active flag constrained to 0 or 1.
users.created_at	TEXT	NOT NULL; DEFAULT CURRENT_TIMESTAMP	Timestamp when the user row was created.
asset_categories.asset_category_id	INTEGER	PK	Surrogate identifier for an asset category.
asset_categories.category_name	TEXT	NOT NULL; UNIQUE	Unique category label, such as Laptop, Printer or Server.
asset_categories.description	TEXT	Nullable	Optional explanation of equipment included in the category.
ticket_categories.ticket_category_id	INTEGER	PK	Surrogate identifier for a support category.
ticket_categories.category_name	TEXT	NOT NULL; UNIQUE	Unique category such as Hardware, Network, Software or Access.
ticket_categories.description	TEXT	Nullable	Optional explanation of the ticket category.
priorities.priority_id	INTEGER	PK	Surrogate identifier for a priority level.
priorities.priority_name	TEXT	NOT NULL; UNIQUE; CHECK: Critical/High/Medium/Low	Unique priority label: Critical, High, Medium or Low.
priorities.sla_hours	INTEGER	NOT NULL; CHECK: > 0	Target number of hours associated with the priority.
priorities.sort_order	INTEGER	NOT NULL; UNIQUE	Unique numeric ordering used to present priorities consistently.
assets.asset_id	INTEGER	PK	Surrogate identifier for an asset record.
assets.asset_tag	TEXT	NOT NULL; UNIQUE	Human-readable inventory tag; unique across assets.
assets.asset_name	TEXT	NOT NULL	Descriptive name used in the inventory and linked-ticket views.
assets.asset_category_id	INTEGER	NOT NULL; FK -> asset_categories.asset_category_id	Required reference to the controlled asset category.

Table.Attribute	SQLite type	Key / constraint	Description
assets.department_id	INTEGER	NOT NULL; FK -> departments.department_id	Department responsible for the asset.
assets.assigned_user_id	INTEGER	FK -> users.user_id	Optional user currently assigned the asset.
assets.manufacturer	TEXT	NOT NULL	Device manufacturer.
assets.model_number	TEXT	NOT NULL	Manufacturer model or model family.
assets.serial_number	TEXT	UNIQUE	Optional manufacturer serial number; unique when provided.
assets.purchase_date	TEXT	NOT NULL; CHECK: valid date	Purchase date stored as an ISO-format date string.
assets.asset_status	TEXT	NOT NULL; CHECK: allowed lifecycle values; DEFAULT 'In Use'	Lifecycle state: In Use, In Repair, In Stock, Retired or Lost.
assets.location	TEXT	NOT NULL; DEFAULT 'Main Campus'	Physical or organisational location of the asset.
assets.notes	TEXT	Nullable	Optional operational notes about the device.
assets.created_at	TEXT	NOT NULL; DEFAULT CURRENT_TIMESTAMP	Timestamp when the asset row was created.
warranties.warranty_id	INTEGER	PK	Surrogate identifier for one warranty record.
warranties.asset_id	INTEGER	NOT NULL; FK -> assets.asset_id	Asset covered by the warranty.
warranties.provider	TEXT	NOT NULL	Warranty provider or supplier name.
warranties.warranty_start	TEXT	NOT NULL; CHECK: valid date	Warranty coverage start date.
warranties.warranty_end	TEXT	NOT NULL; CHECK: valid date and >= start	Warranty expiry date; must not precede the start date.
warranties.coverage_details	TEXT	NOT NULL	Description of covered services or parts.
warranties.reference_number	TEXT	UNIQUE	Optional unique provider claim or warranty reference.
warranties.created_at	TEXT	NOT NULL; DEFAULT CURRENT_TIMESTAMP	Timestamp when the warranty row was created.
tickets.ticket_id	INTEGER	PK	Surrogate primary key used internally by the application.
tickets.ticket_number	TEXT	NOT NULL; UNIQUE	Public human-readable ticket identifier, e.g. HD-26013; unique.
tickets.ticket_type	TEXT	NOT NULL; CHECK: Incident/Service Request	Incident or Service Request; determines the subtype record.
tickets.requester_id	INTEGER	NOT NULL; FK -> users.user_id	Required user who submitted the ticket.
tickets.asset_id	INTEGER	FK -> assets.asset_id	Optional linked asset; becomes NULL if the asset reference is removed.
tickets.ticket_category_id	INTEGER	NOT NULL; FK -> ticket_categories.ticket_category_id	Required support category reference.
tickets.priority_id	INTEGER	NOT NULL; FK -> priorities.priority_id	Required priority reference whose SLA hours inform the due time.
tickets.title	TEXT	NOT NULL; CHECK: trimmed length > 2	Short issue or request summary; trimmed length must exceed two characters.
tickets.description	TEXT	NOT NULL; CHECK: trimmed length > 4	Detailed problem/request description; trimmed length must exceed four characters.
tickets.status	TEXT	NOT NULL; CHECK: allowed ticket states; DEFAULT 'Open'	Ticket state: Open, Assigned, In Progress, On Hold, Resolved or Closed.
tickets.created_at	TEXT	NOT NULL; DEFAULT CURRENT_TIMESTAMP	Timestamp when the ticket was created.
tickets.updated_at	TEXT	NOT NULL; DEFAULT CURRENT_TIMESTAMP	Timestamp of the latest ticket update.
tickets.due_at	TEXT	CHECK: valid date if set	Optional calculated SLA due timestamp.
tickets.resolved_at	TEXT	CHECK: valid date if set	Optional resolution timestamp.

Table.Attribute	SQLite type	Key / constraint	Description
incidents.ticket_id	INTEGER	PK; FK -> tickets.ticket_id	Primary key and foreign key to the parent ticket; one incident subtype per ticket.
incidents.impact	TEXT	NOT NULL; CHECK: Low/Moderate/High/Critical	Business impact level for the incident.
incidents.urgency	TEXT	NOT NULL; CHECK: Low/Moderate/High/Critical	Urgency level recorded for triage.
incidents.occurred_at	TEXT	Nullable	Optional timestamp when the incident occurred or was reported.
incidents.affected_service	TEXT	Nullable	Optional service or business function affected by the incident.
service_requests.ticket_id	INTEGER	PK; FK -> tickets.ticket_id	Primary key and foreign key to the parent service-request ticket.
service_requests.request_type	TEXT	NOT NULL	Kind of service requested.
service_requests.approval_status	TEXT	NOT NULL; CHECK: allowed approval states; DEFAULT 'Not Required'	Approval state: Not Required, Pending, Approved or Rejected.
service_requests.requested_for	TEXT	Nullable	Optional person or beneficiary for whom the request is raised.
support_staff.staff_id	INTEGER	PK	Surrogate identifier for a support staff profile.
support_staff.user_id	INTEGER	NOT NULL; UNIQUE; FK -> users.user_id	Unique reference to the corresponding user identity.
support_staff.staff_code	TEXT	NOT NULL; UNIQUE	Unique internal technician/staff code.
support_staff.specialization	TEXT	NOT NULL	Technical support specialisation.
support_staff.shift_name	TEXT	NOT NULL; DEFAULT 'General'	Shift label; defaults to General.
support_staff.is_available	INTEGER	NOT NULL; DEFAULT 1	Boolean-like availability flag constrained to 0 or 1.
ticket_assignments.assignment_id	INTEGER	PK	Surrogate identifier for one assignment event.
ticket_assignments.ticket_id	INTEGER	NOT NULL; FK -> tickets.ticket_id	Ticket being assigned; a ticket can have multiple historical assignments.
ticket_assignments.staff_id	INTEGER	NOT NULL; FK -> support_staff.staff_id	Support staff member receiving the assignment.
ticket_assignments.assigned_by	INTEGER	NOT NULL; FK -> users.user_id	User who recorded the assignment.
ticket_assignments.assigned_at	TEXT	NOT NULL; DEFAULT CURRENT_TIMESTAMP	Assignment timestamp; defaults to current timestamp.
ticket_assignments.unassigned_at	TEXT	CHECK: end >= assignment timestamp	Nullable timestamp marking when the assignment ended.
ticket_assignments.assignment_note	TEXT	Nullable	Optional handover or assignment note.
ticket_status_history.history_id	INTEGER	PK	Surrogate identifier for one status-history event.
ticket_status_history.ticket_id	INTEGER	NOT NULL; FK -> tickets.ticket_id	Ticket whose status changed.
ticket_status_history.old_status	TEXT	Nullable	Previous status; NULL for the initial Open event.
ticket_status_history.new_status	TEXT	NOT NULL; CHECK: allowed ticket states	New status after the transition; restricted to supported states.
ticket_status_history.changed_by	INTEGER	NOT NULL; FK -> users.user_id	User who is attributed with recording the transition.
ticket_status_history.changed_at	TEXT	NOT NULL; DEFAULT CURRENT_TIMESTAMP	Timestamp of the status event.
ticket_status_history.note	TEXT	Nullable	Optional explanation for the change.
resolutions.resolution_id	INTEGER	PK	Surrogate identifier for a resolution record.
resolutions.ticket_id	INTEGER	NOT NULL; UNIQUE; FK -> tickets.ticket_id	Unique foreign key to the ticket; at most one current resolution row per ticket.
resolutions.resolved_by	INTEGER	NOT NULL; FK -> support_staff.staff_id	Required support staff reference for the technician recording the resolution.

Table.Attribute	SQLite type	Key / constraint	Description
resolutions.resolution_text	TEXT	NOT NULL; CHECK: trimmed length >= 8	Required description of the corrective steps and verification.
resolutions.root_cause	TEXT	Nullable	Optional explanation of the suspected or confirmed cause.
resolutions.resolved_at	TEXT	NOT NULL; DEFAULT CURRENT_TIMESTAMP	Timestamp of the recorded resolution.
resolutions.preventive_action	TEXT	Nullable	Optional action intended to reduce recurrence.
maintenance_records.maintenance_id	INTEGER	PK	Surrogate identifier for the maintenance task or record.
maintenance_records.asset_id	INTEGER	NOT NULL; FK -> assets.asset_id	Required asset whose service history is being recorded.
maintenance_records.performed_by	INTEGER	FK -> support_staff.staff_id	Optional reference to the support staff member performing the task.
maintenance_records.maintenance_type	TEXT	NOT NULL; CHECK: allowed maintenance types	Preventive, Corrective, Inspection or Upgrade.
maintenance_records.scheduled_date	TEXT	NOT NULL; CHECK: valid date	Planned maintenance date.
maintenance_records.completed_date	TEXT	CHECK: valid date if set	Optional completion date.
maintenance_records.cost	REAL	NOT NULL; CHECK: >= 0; DEFAULT 0	Recorded cost value; non-negative and defaults to zero.
maintenance_records.status	TEXT	NOT NULL; CHECK: allowed maintenance states; DEFAULT 'Scheduled'	Scheduled, In Progress, Completed or Cancelled.
maintenance_records.notes	TEXT	Nullable	Task description, service action or inspection note.
maintenance_records.created_at	TEXT	NOT NULL; DEFAULT CURRENT_TIMESTAMP	Timestamp when the maintenance row was created.

2.6 Implementation of Logical Design: DDL and Sample Data

The Flask application creates the local database at `instance/helpdesk.db` on first launch and loads the DDL from `sql/schema_sqlite.sql`. The runtime connection uses Python's `sqlite3` module, sets rows to `sqlite3.Row`, enables foreign-key enforcement and sets a busy timeout. Database writes that involve related rows use connection context managers so grouped operations commit together or roll back on an exception. The project also includes a MySQL 8 schema reference at `sql/mysql_schema.sql`; this reference is not connected to the live prototype [3]-[5].

Table 9. Database Implementation Environment

Item	Details
DBMS	SQLite (implemented runtime DBMS)
Database file	`instance/helpdesk.db` in local project
Declared schema tables	15
Declared columns	104 across 15 tables
Schema source	`sql/schema_sqlite.sql`
Reference migration	`sql/mysql_schema.sql` (not the active connector)
Runtime data	Synthetic demo departments, users, assets, tickets, assignments, warranties and maintenance rows

Representative DDL

```
CREATE TABLE IF NOT EXISTS tickets (
    ticket_id INTEGER PRIMARY KEY AUTOINCREMENT,
    ticket_number TEXT NOT NULL UNIQUE,
    ticket_type TEXT NOT NULL
        CHECK (ticket_type IN ('Incident','Service Request')),
    requester_id INTEGER NOT NULL,
    asset_id INTEGER,
    ticket_category_id INTEGER NOT NULL,
    priority_id INTEGER NOT NULL,
    title TEXT NOT NULL CHECK (length(trim(title)) > 2),
    description TEXT NOT NULL CHECK (length(trim(description)) > 4),
    status TEXT NOT NULL DEFAULT 'Open'
        CHECK (status IN ('Open','Assigned','In Progress',
                          'On Hold','Resolved','Closed')),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    updated_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    due_at TEXT,
    resolved_at TEXT,
    FOREIGN KEY (requester_id) REFERENCES users(user_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    FOREIGN KEY (asset_id) REFERENCES assets(asset_id)
        ON UPDATE CASCADE ON DELETE SET NULL,
    FOREIGN KEY (ticket_category_id)
        REFERENCES ticket_categories(ticket_category_id)
        ON UPDATE CASCADE ON DELETE RESTRICT,
    FOREIGN KEY (priority_id) REFERENCES priorities(priority_id)
        ON UPDATE CASCADE ON DELETE RESTRICT
);
```

A ticket insert is followed by a subtype insert into either `incidents` or `service_requests` and an initial `ticket_status_history` event. The operation is grouped in one database context. The due timestamp is calculated from the selected priority's SLA hours. A synthetic manual test created ticket HD-26013 for Rahul Verma, linked it to asset AST-1001, assigned it to Vikram Shah, moved it to In Progress and recorded a resolution. This sequence is evidenced by the screenshots and exported CSV; it is not a statement about a real finance laptop or production incident.

Representative sample record from the manual demonstration

Field	Observed value
Ticket number / title	HD-26013 — Finance laptop intermittently loses network connection
Type / category / priority	Incident / Network / High (4-hour SLA)
Requester / department	Rahul Verma / Finance
Linked asset	AST-1001 — Finance Laptop (Dell Latitude 5440)
Assignment	Vikram Shah — Network & Infrastructure
Lifecycle	Open → In Progress → Resolved
Resolution record	Resolution text and timestamp displayed on ticket detail page
Data status	Synthetic academic demonstration record

CHAPTER 3
APPLICATION DEVELOPMENT USING LOGICAL DATABASE DESIGN

3.1 Application Overview

Helix IT is a local browser-based application for helpdesk and asset support operations. The main interface is generated by Flask/Jinja templates, while CSS and vanilla JavaScript provide the navigation, forms and responsive table presentation. The application is connected to the SQLite schema described in Chapter 2 and uses the same relational records for dashboard summaries, ticket details, asset history, maintenance views and reporting outputs. The interface does not use production identity services; actions in the prototype are attributed to a seeded administrator for demonstration purposes.

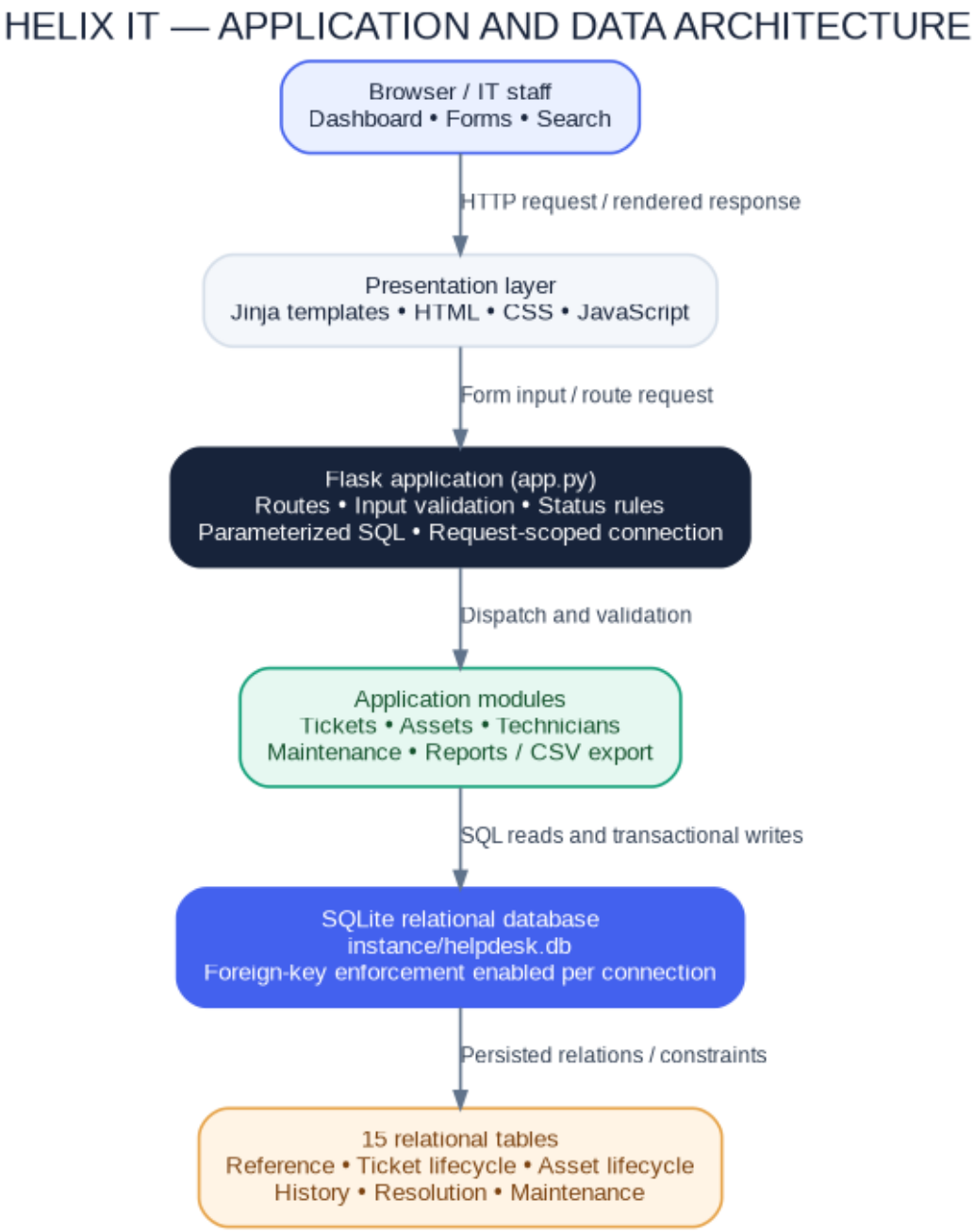


Figure 2. Application and Data Architecture

3.2 Development Environment and Database Connectivity

The prototype was manually launched on macOS in a Python 3.13 environment with Flask 3.1.3 installed. The browser connected to the local address `http://127.0.0.1:5000`. The screenshot evidence shows successful HTTP responses for the dashboard, stylesheets, JavaScript, asset inventory, maintenance, reports and CSV route. Exact SQLite runtime patch version was not captured and is therefore not asserted here [5], [6].

Table 10. Application Development Environment

Item	Details
Programming language	Python (observed runtime: Python 3.13 series)
Web framework	Flask 3.1.3 was shown in the local installation output

Item	Details
Database	SQLite, accessed using Python `sqlite3`
Frontend	HTML5, Jinja templates, CSS, vanilla JavaScript
Repository	https://github.com/shivaganesh2579/IT-Helpdesk-Asset-Support-Management
Launch command	`python app.py` from the project directory
Local address	`http://127.0.0.1:5000`
Test framework	Python `unittest` test module present in `tests/test_app.py`; execution remains pending verification

3.3 Application Modules and Database Mapping

Table 11. Application Module-to-Database Mapping

Module / screen	Purpose	Tables used
Overview dashboard	Ticket backlog, urgent work, asset count, warranty watch and activity.	tickets, priorities, assets, warranties, ticket_assignments, support_staff, users
Tickets	Create/search/filter tickets; view details; assign staff; change status; record resolution.	tickets, incidents, service_requests, users, departments, assets, ticket_categories, priorities, ticket_assignments, ticket_status_history, resolutions
Asset inventory	List/search/filter/register/edit assets and view linked support history.	assets, asset_categories, departments, users, warranties, tickets, maintenance_records
Technicians	Show technician identity, specialisation, availability, active/resolved workload.	support_staff, users, ticket_assignments, tickets, resolutions
Maintenance	Add maintenance tasks and update status/completion date; display cost summaries.	maintenance_records, assets, support_staff, users
Reports & analytics	Open-ticket ageing, average first assignment, technician workload, asset history, warranty-expiry, recurring categories.	tickets, ticket_assignments, support_staff, users, assets, maintenance_records, warranties, priorities, ticket_categories
CSV export / search	Export ticket report columns; search ticket number/title/status and asset information.	tickets, priorities, ticket_categories, users, assets

3.4 CRUD Operations

Table 12. CRUD Operations

Operation	Application feature / screen	Tables affected and behavior
Create	New ticket form	INSERT into tickets and corresponding incidents/service_requests; add first ticket_status_history event.
Read	Dashboard, tickets, ticket detail, reports	SELECT with joins, calculated fields and aggregate subqueries.
Update	Assignment, status, resolution, asset edit, maintenance status	UPDATE parent row and insert corresponding history/resolution/assignment record inside database transaction contexts.
Delete / controlled removal	Ticket or asset remove routes	Ticket deletion is restricted to eligible new Open tickets without assignment history. Assets referenced by tickets should be retired rather than deleted.

3.5 Business Rules, Validation and Error Handling

Table 13. Business Rules, Validation and Error Handling

Rule / validation	How implemented	Evidence / expected behavior
Ticket title and description	Route validates trimmed minimum lengths; DDL also has CHECK constraints.	The form is re-rendered with an error message if required text is too short.

Rule / validation	How implemented	Evidence / expected behavior
Ticket and asset identity	UNIQUE constraints on ticket_number, asset_tag and applicable natural identifiers.	Duplicate insert raises an integrity error and is handled by route-level error feedback.
Referential integrity	SQLite foreign keys are enabled on each request connection.	Invalid references are rejected by the database.
Status transitions	Python route checks the current state and allowed next states; generic form cannot resolve a ticket without the resolution workflow.	Invalid transition is rejected with feedback and no state update.
Resolution rules	Resolution text must have at least 8 characters and resolving technician must exist. Closing requires prior resolution.	Resolution details and the ticket's Resolved state are saved as related updates.
Date and cost validation	Date CHECKs and application parsing; warranty end cannot precede start; maintenance cost must be >= 0.	Invalid values are rejected by schema or route handling.
Safe query parameters	User-supplied values are passed through SQLite `?` placeholders instead of being concatenated into SQL.	Reduces risk from value-based SQL injection and preserves correct quoting.
Transactions and history	Multi-row operations use connection context managers; assignment/status/resolution changes preserve event records.	Related writes commit together or roll back on an exception.

3.6 Reports and Database Operations

Table 14. Reports and Database Operations

Report / operation	SQL/database concepts	Application output
Open-ticket ageing	JOIN tickets to users and priorities; calculated age using SQLite `julianday`; ordering by age and priority.	Open-ticket table with requester, owner, created date, age and status.
First-assignment time	LEFT JOIN to subquery using `MIN(assigned_at)` per ticket; `AVG` and time difference.	Average first-assignment metric shown in dashboard.
SLA overdue count	Filter unresolved statuses and compare `due_at` to current time.	Overdue open-ticket card and due-date monitoring.
Technician workload	JOIN assignments, support staff, users and tickets; conditional `COUNT(DISTINCT ...)` and `GROUP BY`.	Active and resolved ticket counts by technician.
Asset history	LEFT JOIN assets, tickets and maintenance; distinct aggregate counts and latest ticket date.	Ticket count and maintenance count per asset.
Warranty expiry	JOIN warranties to assets and calculate days remaining; filter dates through the next 90 days.	Warranty-expiry watch for upcoming and expired warranty records.
Recurring categories	JOIN tickets to ticket_categories; aggregate ticket/open counts and affected assets; `HAVING COUNT(*) >= 2`.	Categories with repeated tickets.
CSV export	Join tickets with priorities, categories and users; Python `csv.writer` outputs fixed column headers.	Downloaded `helpdesk-ticket-report.csv` listing ticket number, type, title, state, priority, category, requester and timestamps.

The exported CSV includes ticket HD-26013 with Resolved status and a resolution timestamp. Analytics are generated from synthetic demo records and can change as records or the current date change. Maintenance cost totals include recorded values for scheduled as well as completed tasks, so the card is not an audited measure of cash already spent.

3.7 Application Screens

Figures 3–14 provide the main visual evidence from the local browser demonstration. The screenshots document data entry, ticket lifecycle updates, inventory search, maintenance, analytics and CSV export. The displayed names and ticket records are synthetic demonstration data.

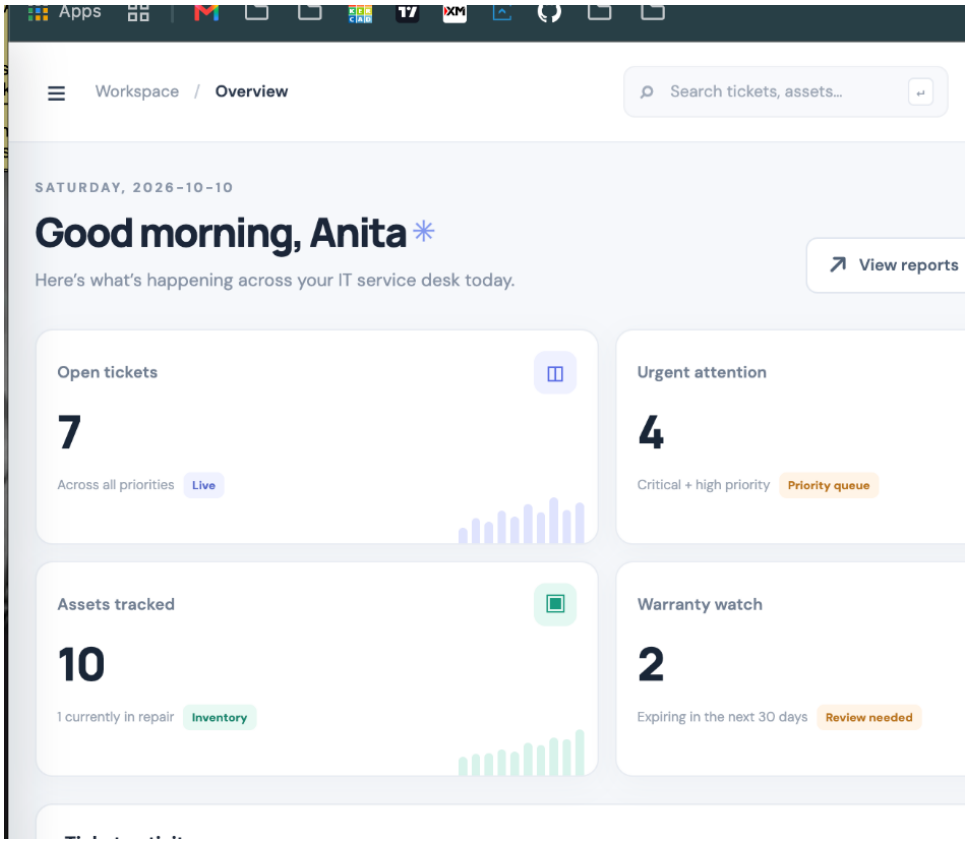


Figure 3. Overview Dashboard

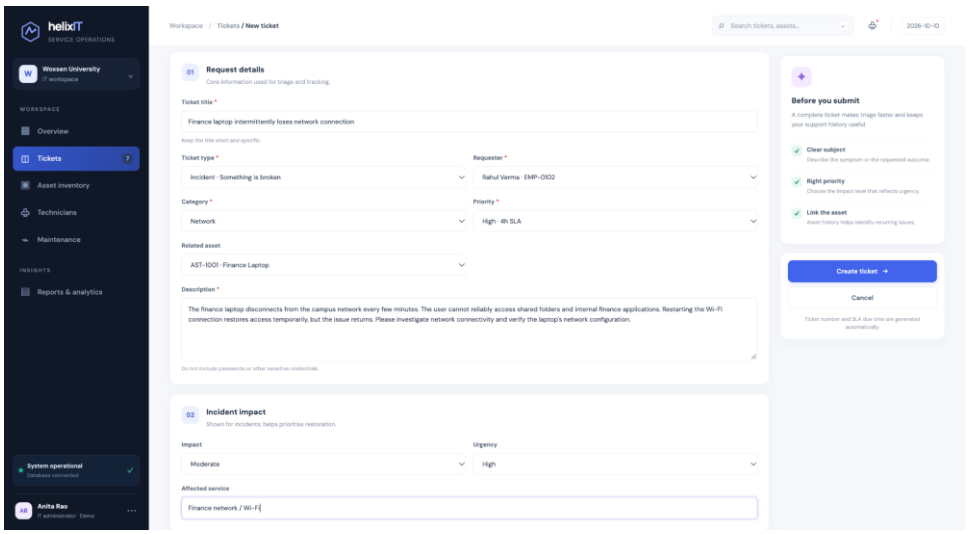


Figure 4. New Ticket Form with Incident Details

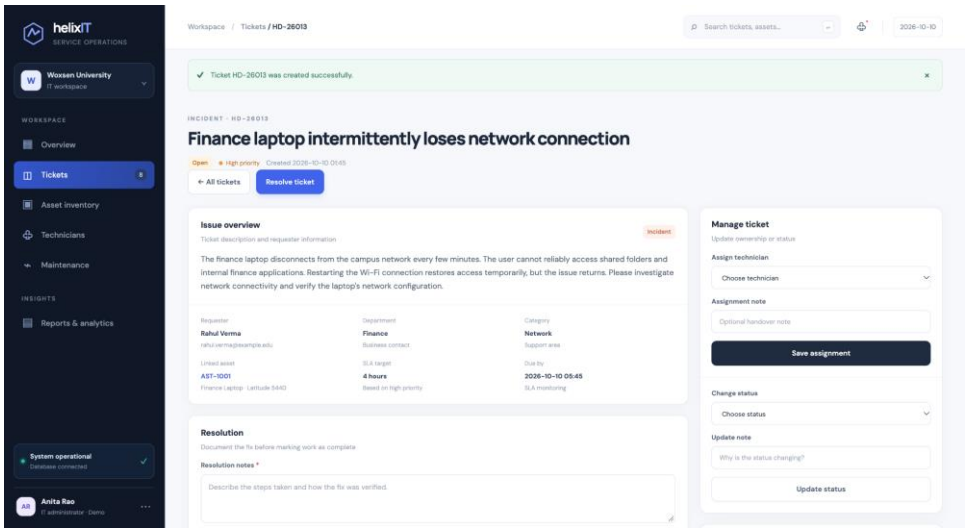


Figure 5. Ticket HD-26013 Created Successfully

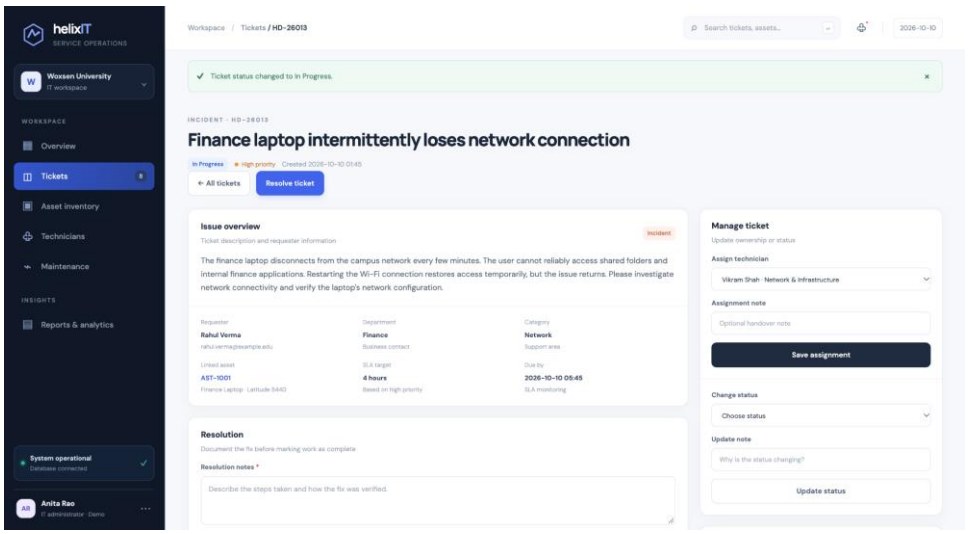


Figure 6. Ticket Status Updated to In Progress

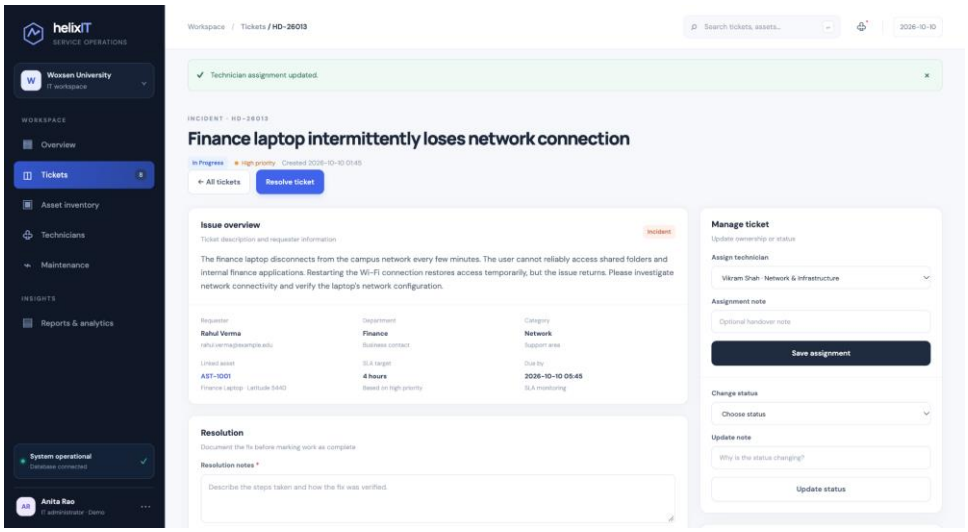


Figure 7. Technician Assignment Confirmation

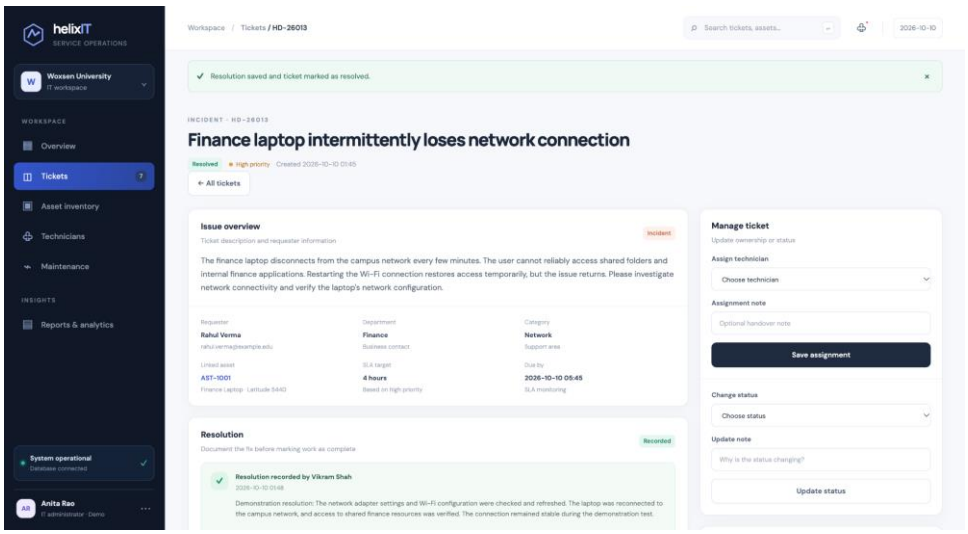


Figure 8. Ticket HD-26013 Resolved with Recorded Resolution

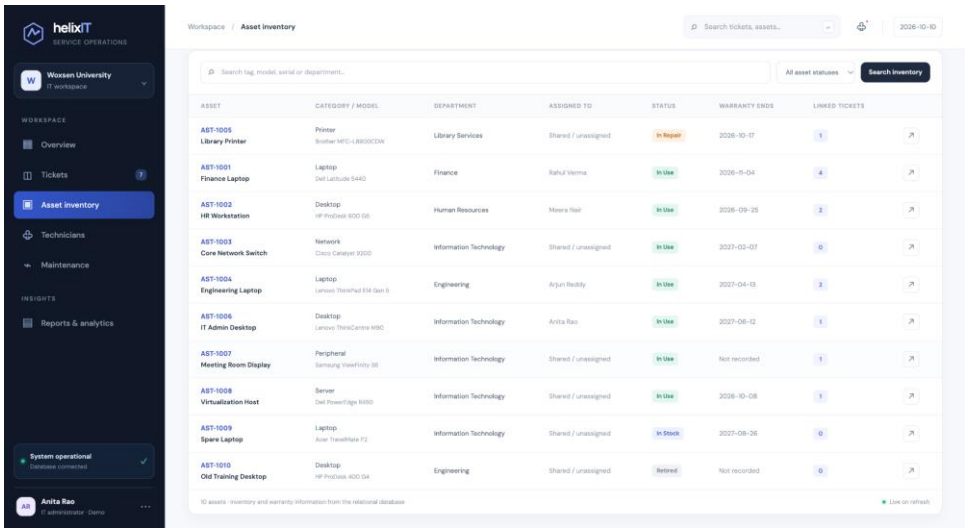


Figure 9. Asset Inventory

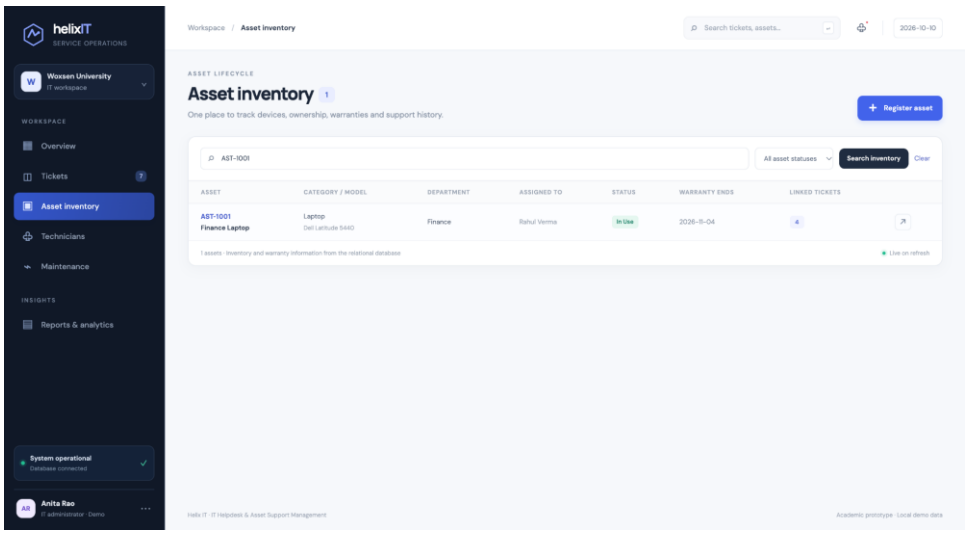


Figure 10. Asset Search Result for AST-1001

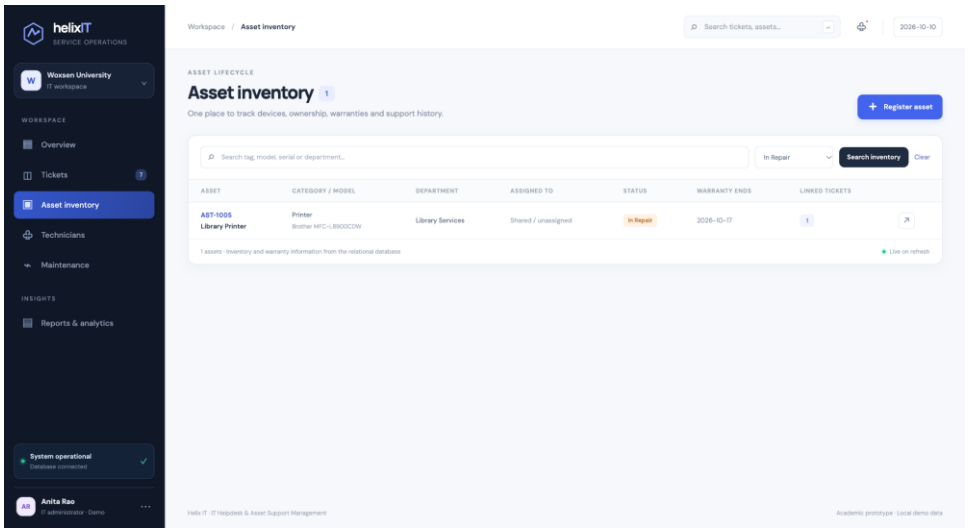


Figure 11. Asset Status Filter: In Repair

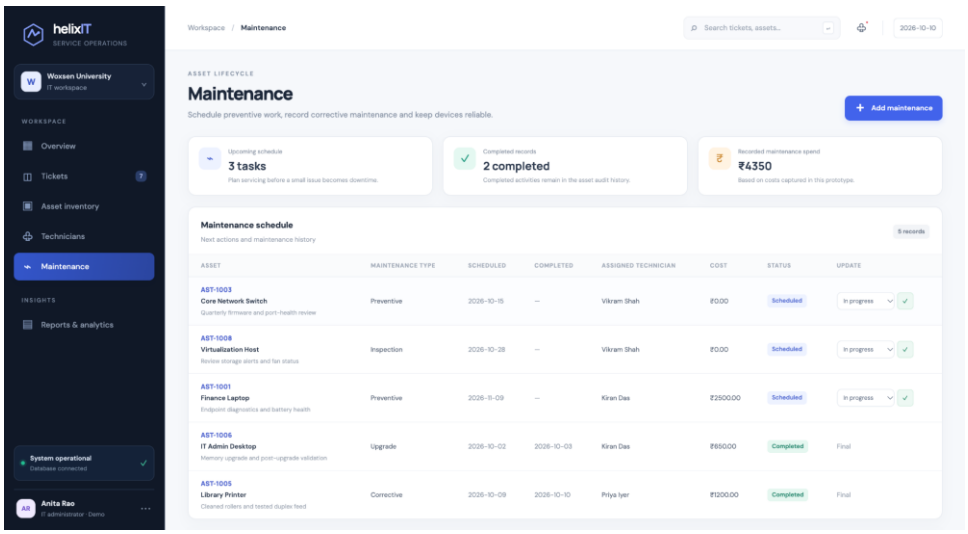


Figure 12. Maintenance Schedule and Recorded Costs

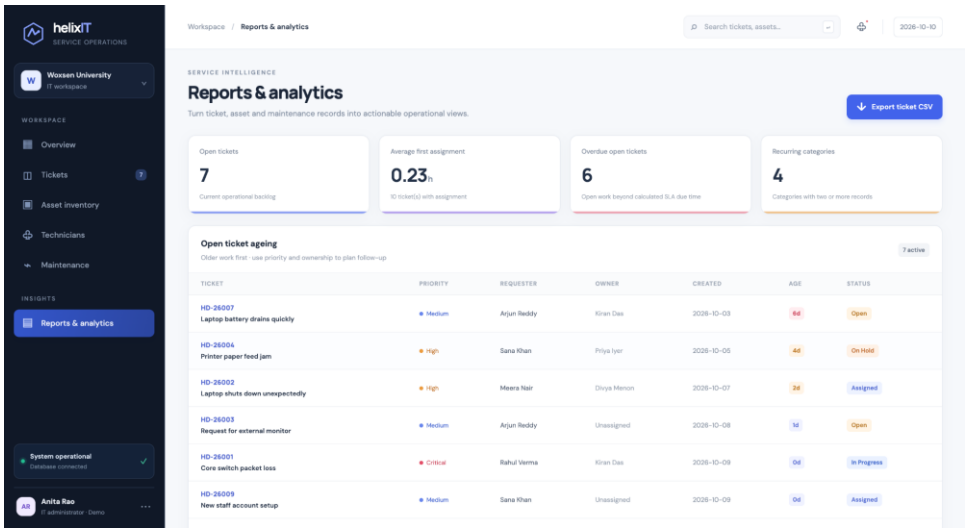


Figure 13. Reports and Analytics Dashboard

helpdesk-ticket-report

Ticket Number	Type	Title	Status	Priority	Category	Requester	Created At	Due At	Resolved At
HD-26010	Incident	Internal support portal unavailable	In Progress	Critical	Network	Nikhil Bose	2026-10-10 11:00:00	2026-10-10 13:00:00	
HD-26013	Incident	Finance laptop intermittently loses network connection	Resolved	High	Network	Rahul Verma	2026-10-10 01:45:18	2026-10-10 05:45:18	2026-10-10 01:48:59
HD-26009	Service Request	New staff account setup	Assigned	Medium	Asset Request	Sana Khan	2026-10-09 10:00:00	2026-10-09 18:00:00	
HD-26001	Incident	Core switch packet loss	In Progress	Critical	Hardware	Rahul Verma	2026-10-09 09:00:00	2026-10-09 11:00:00	
HD-26012	Service Request	Install approved PDF software	Resolved	Low	Asset Request	Meera Nair	2026-10-08 13:00:00	2026-10-09 13:00:00	2026-10-09 14:00:00
HD-26003	Service Request	Request for external monitor	Open	Medium	Asset Request	Arjun Reddy	2026-10-08 11:00:00	2026-10-08 19:00:00	
HD-26002	Incident	Laptop shuts down unexpectedly	Assigned	High	Software	Meera Nair	2026-10-07 10:00:00	2026-10-07 14:00:00	
HD-26005	Incident	VPN client fails to connect	Resolved	Medium	Software	Kiran Das	2026-10-06 13:00:00	2026-10-06 21:00:00	2026-10-07 14:00:00
HD-26004	Incident	Printer paper feed jam	On Hold	High	Hardware	Sana Khan	2026-10-05 12:00:00	2026-10-05 16:00:00	
HD-26007	Incident	Laptop battery drains quickly	Open	Medium	Software	Arjun Reddy	2026-10-03 15:00:00	2026-10-03 23:00:00	
HD-26008	Incident	Email sync delay	Resolved	Medium	Email & Collaboration	Meera Nair	2026-10-02 09:00:00	2026-10-02 17:00:00	2026-10-03 14:00:00
HD-26006	Service Request	Shared mailbox access	Closed	Low	Access	Rahul Verma	2026-09-28 14:00:00	2026-09-29 14:00:00	2026-09-29 14:00:00
HD-26011	Incident	Network connectivity drop	Closed	High	Hardware	Rahul Verma	2026-09-20 12:00:00	2026-09-20 16:00:00	2026-09-21 14:00:00

Figure 14. Exported Ticket CSV Report

3.8 Testing and Results

The result table separates browser workflows observed during the student demonstration from automated tests that are present in the repository but have not been executed in the report-authoring environment. Manual "Pass" entries reflect the browser screens and exported CSV supplied for this report. Pending automated tests must be run on the student's configured Mac before the final submission claims those tests passed [7].

Table 15. Test Cases

Test ID	Feature	Input / action	Expected result	Actual result / evidence	Status
TC-01	Dashboard / connection	Open `http://127.0.0.1:5000`.	Dashboard displayed summary cards and "Database connected" state.	Dashboard loaded in browser.	Pass - manual
TC-02	Ticket creation	Submit Finance Laptop incident form.	New unique ticket and success confirmation.	Ticket HD-26013 created; linked asset AST-1001.	Pass - manual
TC-03	Technician assignment	Assign Vikram Shah and save.	Assignment confirmation and technician information.	"Technician assignment updated" was displayed.	Pass - manual
TC-04	Status update	Change ticket from Open/Assigned to In Progress.	Status is updated and history is retained.	"Ticket status changed to In Progress" was displayed.	Pass - manual
TC-05	Resolution	Enter resolution text and select Resolve.	Resolution record is saved and ticket shows Resolved.	Success banner and resolution note were visible.	Pass - manual
TC-06	Asset search	Search for AST-1001.	Finance Laptop appears as the matching asset.	One matching AST-1001 row appeared.	Pass - manual
TC-07	Status filter	Filter asset inventory to In Repair.	Only matching lifecycle records appear.	Library Printer AST-1005 appeared.	Pass - manual
TC-08	Maintenance page	Open Maintenance.	Maintenance records and cost summary load.	Five rows and ₹4,350 recorded cost summary displayed.	Pass - manual
TC-09	Reports page	Open Reports & analytics after correcting missing selected ticket_id.	Analytics cards and ageing report render.	Reports page loaded and showed records.	Pass - manual after fix
TC-10	CSV export	Click Export ticket CSV.	CSV downloads with ticket columns and current data.	Exported report contained HD-26013 with Resolved state/time.	Pass - manual
TC-11	Duplicate asset tag	Run automated duplicate AST-1001 registration test in `tests/test_app.py`.	Duplicate is rejected with controlled error feedback.	Automated execution not evidenced in this report.	Pending automated run
TC-12	Invalid status transition	Run test for a prohibited status transition in `tests/test_app.py`.	Transition is rejected and prior status remains.	Automated execution not evidenced in this report.	Pending automated run
TC-13	Automated suite	Run `python -m unittest discover -s tests -v` on configured environment.	All included tests execute and results are captured.	Not executed in report-authoring environment because Flask was unavailable there.	Pending execution

The manual demonstrations establish that the ticket lifecycle, asset search/filter, maintenance list, report rendering after the query/template correction and CSV export were exercised in the browser. The automated suite file includes route and data-integrity tests, but its execution status remains pending in this document. Before final submission, run the test command in the same environment used to demonstrate the application, save the full terminal output and update the pending rows to the observed outcomes. The tests use a temporary database by design and should not overwrite the student's local demo database.

CHAPTER 4
CONCLUSION

This project designed and implemented a relational database-backed IT Helpdesk and Asset Support Management prototype. The logical model separates user identity, departments, asset and ticket classifications, priority/SLA definitions, ticket subtypes, assignment events, status history, resolutions, warranty details and maintenance records. The SQLite schema declares 15 tables and 104 columns, with primary keys, foreign keys, unique identifiers, CHECK constraints, defaults and indexes for commonly queried attributes. The Flask interface connects to that schema and provides ticket, asset, technician, maintenance, reporting and CSV-export workflows.

The manual demonstration created ticket HD-26013, linked it to Finance Laptop AST-1001, assigned a technician, moved the ticket to In Progress, recorded a resolution and verified that the Resolved record appeared in the exported report. Asset inventory search/filter and the reports page were also demonstrated. The project therefore illustrates the relationship between conceptual ER modelling, relational mapping, normalization, SQL joins and aggregates, transaction-aware application writes and user-facing operations. The current implementation remains an academic local prototype; the final submission should include the completed report fields and a captured automated test run.

4.1 Limitations and Future Enhancements

- Add authentication, role-based access control and secure session/configuration management before exposing the application beyond a trusted local demonstration.
- Run and record the complete automated test suite; add more negative tests for invalid dates, missing related records, prohibited lifecycle transitions and duplicate business identifiers.
- Improve reporting semantics by separating planned maintenance cost from completed actual cost and by documenting how SLA metrics treat resolved or unassigned tickets.
- Add backup/restore procedures, audit export, pagination, stronger date/time-zone handling and a deployment configuration suitable for a controlled server.
- If required by faculty, migrate the active connector from SQLite to MySQL using `sql/mysql_schema.sql`, then re-run all data-integrity and application tests against MySQL before describing it as the runtime DBMS [8].
- Add notifications and configurable SLAs only as a future capability; neither email notifications nor production integrations are implemented in the current prototype.

REFERENCES

[1] A. Silberschatz, H. F. Korth, and S. Sudarshan, Database System Concepts, 7th ed. New York, NY, USA: McGraw-Hill, 2019. [Online]. Available: https://www.db-book.com/.

[2] Mermaid, “Entity Relationship Diagrams,” Mermaid Documentation. Accessed: Oct. 10, 2026. [Online]. Available: https://mermaid.js.org/syntax/entityRelationshipDiagram.html.

[3] SQLite, “CREATE TABLE,” SQLite Documentation. Accessed: Oct. 10, 2026. [Online]. Available: https://www.sqlite.org/lang_createtable.html.

[4] SQLite, “SQLite Foreign Key Support,” SQLite Documentation. Accessed: Oct. 10, 2026. [Online]. Available: https://www.sqlite.org/foreignkeys.html.

[5] Python Software Foundation, “sqlite3 — DB-API 2.0 interface for SQLite databases,” Python 3.13 Documentation. Accessed: Oct. 10, 2026. [Online]. Available: https://docs.python.org/3.13/library/sqlite3.html.

[6] Pallets Projects, “Quickstart,” Flask Documentation, 3.1.x. Accessed: Oct. 10, 2026. [Online]. Available: https://flask.palletsprojects.com/en/stable/quickstart/.

[7] Pallets Projects, “Testing Flask Applications,” Flask Documentation, 3.1.x. Accessed: Oct. 10, 2026. [Online]. Available: https://flask.palletsprojects.com/en/stable/testing/.

[8] Oracle, “MySQL 8.4 Reference Manual: CREATE TABLE Statement,” MySQL Documentation. Accessed: Oct. 10, 2026. [Online]. Available: https://dev.mysql.com/doc/refman/8.4/en/create-table.html.

CHAPTER A
COMPLETE DATABASE SCRIPT

A.1 Implemented SQLite DDL: `sql/schema_sqlite.sql`

The complete, executable SQLite DDL is maintained as `sql/schema_sqlite.sql` in the submitted repository. The excerpt below illustrates representative keys, foreign keys and CHECK constraints; the full schema contains all 15 implemented tables and supporting indexes documented in Chapter 2.

```
PRAGMA foreign_keys = ON;

CREATE TABLE IF NOT EXISTS departments (
    department_id INTEGER PRIMARY KEY AUTOINCREMENT,
    department_name TEXT NOT NULL UNIQUE,
    location TEXT NOT NULL DEFAULT 'Main Campus'
);

CREATE TABLE IF NOT EXISTS tickets (
    ticket_id INTEGER PRIMARY KEY AUTOINCREMENT,
    ticket_number TEXT NOT NULL UNIQUE,
    ticket_type TEXT NOT NULL CHECK (ticket_type IN ('Incident','Service Request')),
    requester_id INTEGER NOT NULL,
    asset_id INTEGER,
    ticket_category_id INTEGER NOT NULL,
    priority_id INTEGER NOT NULL,
    title TEXT NOT NULL CHECK (length(trim(title)) > 2),
    description TEXT NOT NULL CHECK (length(trim(description)) > 4),
    status TEXT NOT NULL DEFAULT 'Open' CHECK (status IN ('Open','Assigned','In Progress','On Hold','Resolved','Closed')),
    created_at TEXT NOT NULL DEFAULT CURRENT_TIMESTAMP,
    due_at TEXT,
    resolved_at TEXT,
    FOREIGN KEY (requester_id) REFERENCES users(user_id) ON DELETE RESTRICT,
    FOREIGN KEY (asset_id) REFERENCES assets(asset_id) ON DELETE SET NULL,
    FOREIGN KEY (ticket_category_id) REFERENCES ticket_categories(ticket_category_id) ON DELETE RESTRICT,
    FOREIGN KEY (priority_id) REFERENCES priorities(priority_id) ON DELETE RESTRICT,
    CHECK (due_at IS NULL OR date(due_at) IS NOT NULL),
    CHECK (resolved_at IS NULL OR date(resolved_at) IS NOT NULL)
);

CREATE INDEX IF NOT EXISTS idx_tickets_status_created ON tickets(status, created_at DESC);
```

A.2 Representative Reporting Queries: `sql/report_queries.sql`

The full `sql/report_queries.sql` file in the repository contains nine reporting queries. The examples below illustrate joins and calculated ticket age, aggregate first-assignment time, and warranty-expiry filtering. The complete runnable script remains available in the repository and is the authoritative query listing.

```
-- 1. Open ticket queue and ageing (joins, calculated field, ordering)
SELECT t.ticket_number, t.title, t.status, p.priority_name, u.full_name AS requester,
       CAST(julianday('now') - julianday(t.created_at) AS INTEGER) AS age_days
FROM tickets t
JOIN users u ON u.user_id = t.requester_id
JOIN priorities p ON p.priority_id = t.priority_id
WHERE t.status NOT IN ('Resolved', 'Closed')
ORDER BY age_days DESC, p.sort_order;

-- 2. Average time from ticket creation to first assignment
SELECT ROUND(AVG((julianday(first_assignment.first_assigned) - julianday(t.created_at)) * 24), 2)
       AS avg_first_assignment_hours
FROM tickets t
LEFT JOIN (
    SELECT ticket_id, MIN(assigned_at) AS first_assigned
    FROM ticket_assignments GROUP BY ticket_id
) first_assignment ON first_assignment.ticket_id = t.ticket_id;

-- 3. Warranty expiry within the next 90 days
SELECT a.asset_tag, a.asset_name, w.provider, w.warranty_end,
       CAST(julianday(w.warranty_end) - julianday('now') AS INTEGER) AS days_remaining
FROM warranties w
JOIN assets a ON a.asset_id = w.asset_id
WHERE date(w.warranty_end) <= date('now', '+90 day')
ORDER BY w.warranty_end;
```

CHAPTER B
APPLICATION SOURCE CODE

B.1 Repository and source files

The complete application source is maintained in the public GitHub repository:

https://github.com/shivaganesh2579/IT-Helpdesk-Asset-Support-Management

The repository contains `app.py`, `requirements.txt`, `README.md`, the SQLite schema, a MySQL 8 reference schema, reporting queries, database design notes, ER diagram source, Jinja templates, static CSS/JavaScript, the automated test file, three phase presentations and captured browser evidence. The application should be launched with `python app.py` from the project directory after dependencies are installed. The local `instance/helpdesk.db` file is generated during initialization and is excluded from source control so that local demonstration records are not included in the repository by default.

B.2 Reproduction steps

1. Create a Python virtual environment and activate it.
2. Install dependencies using `python -m pip install -r requirements.txt`.
3. Start the application with `python app.py`.
4. Open `http://127.0.0.1:5000` and follow the demo workflow in the README.
5. Run the automated suite using `python -m unittest discover -s tests -v`; record actual output before claiming the suite passed.

CHAPTER C
ADDITIONAL TEST DATA AND OUTPUTS

C.1 Demonstration record summary

The screenshots and CSV export capture the following representative synthetic record. These values support traceability between the browser interface and the database-backed output.

Supplementary demonstration record

Attribute	Observed value
Ticket number	HD-26013
Ticket title	Finance laptop intermittently loses network connection
Requester	Rahul Verma (Finance)
Asset	AST-1001 — Finance Laptop; Dell Latitude 5440
Category / priority	Network / High (4-hour SLA)
Assigned technician	Vikram Shah — Network & Infrastructure
Final ticket state	Resolved
CSV export	`helpdesk-ticket-report.csv`; row includes resolved timestamp

C.2 Automated test follow-up

The project includes tests for dashboard and health response, ticket list/report rendering, ticket creation and status-history insertion, duplicate asset-tag handling, invalid status-transition rejection and CSV export. This report does not represent these automated tests as passed because a runnable Flask test environment was unavailable during report authoring. Execute the documented command on the student's configured environment and attach the actual terminal output before submission. Replace the pending statuses in Table 15 with the observed results after execution.

C.3 Final student checklist

- Fill in roll number, section, faculty name, contact details and final submission date on the cover and undertaking.
- Review the report wording against the final committed GitHub version, especially the reports query/template correction.
- Run the complete automated test suite and save actual output; do not state that pending tests passed.
- Refresh the Table of Contents and lists if edits or page changes are made in Word.
- Confirm the submitted PDF visually matches the editable DOCX.
- Ensure the final report and required presentations/video link document are committed to the public GitHub repository.